

iTantra: An Offline, On-Device Multilingual Neural Speech Transceiver for Infrastructure-Independent Voice Communication

(Motivated by the ISRO requirement for indigenous, multilingual, low-bitrate voice communication systems)

Sarthak Patil, Utkarsh Vidwat, Gaurav Chaudhari, Ganesh Beldar, Siddhi Bhosale, Soniya Pangatte

Department of Computer Science and Engineering, PCET's Nutan Maharashtra Institute of Engineering and Technology, Pune, Maharashtra, India

Abstract

Cellular towers, the internet, and even grid power tend to fail at exactly the moments when communication matters most — during floods, earthquakes, landslides, and cyclones — and the same infrastructure is often simply absent in tactical field operations, remote rural areas, and mission-support settings. This paper presents iTantra, an offline, on-device Android system built to keep voice communication working under precisely these zero-connectivity conditions. Instead of streaming audio, iTantra converts speech to text on the device itself using AI4Bharat's IndicConformer speech-recognition models, sends only a small structured message (roughly 50 to 300 bytes) over ad-hoc Wi-Fi Direct or Bluetooth, and reconstructs natural-sounding speech on the receiving device using an offline neural voice. The internet is used exactly once, for the initial download of model files, after which every conversation runs with zero network calls. We describe the system as it actually stands rather than as originally planned: speech recognition currently covers nine Indian languages, verified offline voice synthesis covers five of them, and a neural voice-activity detector that failed under real-world testing has been honestly replaced with a simpler energy-based one. A companion on-device assistant, built on a small local language model, answers first-aid and disaster-protocol questions and can translate text between languages, independently of the real-time voice link. We situate this work against existing offline peer-to-peer messaging systems and low-bitrate speech-coding research, lay out the system's architecture and the engineering findings that shaped it, and propose an evaluation plan covering bandwidth savings, recognition accuracy, latency, network performance, resource use, and noise robustness. We close by examining where iTantra fits into India's disaster-management and defence communication priorities, and what would need to change before it could be trusted with that role.

Keywords: Speech-to-Text, Text-to-Speech, Offline Communication, Peer-to-Peer Networking, Multilingual Speech Processing, On-Device Language Models, Disaster Management, ISRO

I. Introduction

Cellular networks and internet infrastructure are built on an assumption that's easy to forget until it fails: continuous power and an intact physical network. During floods, earthquakes, landslides, and cyclones, that assumption breaks down quickly — towers lose power, cables snap, and simply owning a smartphone no longer means you can call for help. The same gap shows up by design in other settings too: tactical field operations, rural areas with patchy GSM coverage, and mission-support scenarios where a conventional network either doesn't reach or isn't meant to be relied upon. A standard voice call reduces to Speech → Audio data → Network → Playback, and every step after the microphone assumes a network able to carry a continuous, fairly heavy stream of audio. Take that network away and the whole chain collapses, no matter how capable the phone itself is.

iTantra inverts this pipeline: Speech → Local STT → Text → Small data packet → Local TTS → Speech. The idea rests on a simple observation — the linguistic content of an utterance is a tiny fraction of the acoustic information carried in a voice. If speech is converted to text on the device itself, and only the transcript, along with a little language metadata, is sent across, the payload shrinks from a continuous audio stream to somewhere around 50 to 300 bytes per utterance. That leads to the question this paper is built around: can on-device speech-to-text and text-to-speech processing carry a real conversation over an extremely low-bandwidth, infrastructure-free peer-to-

peer link, quickly enough to feel usable, and across the languages spoken in India?

We answer this with a working Android implementation, and we describe it honestly, including the parts that did not go according to the original plan. On-device speech recognition runs through AI4Bharat's IndicConformer models, exported per language and served through sherpa-onnx; speech synthesis uses real, espeak-ng-phonemized neural voices (from the Piper, Coqui, and Mimic3 projects) through the same sherpa-onnx runtime, rather than a custom Indic voice model as originally envisioned; and messages are packed into Protocol Buffers and sent over Wi-Fi Direct, with Bluetooth Classic RFCOMM as an automatic fallback. The internet is used exactly once, to download model files on first setup — every subsequent conversation, and every VAD, STT, and TTS inference, happens with zero network calls. A separate on-device assistant, built around a small local language model, answers first-aid and disaster-protocol questions and can translate text across languages, though this sits alongside the real-time voice link rather than inside it. This paper's contributions are: a research framing of on-device, linguistic-channel communication as an alternative to sending raw audio; a system architecture whose network and wire-format choices are backed by measured benchmarks; an honest account of what changed between the original design and the working system, including a documented voice-activity-detection failure and a real gap in text-to-speech language coverage; a comparison against existing offline communication tools that shows where the

actual research contribution lies; and a concrete plan for evaluating the system end to end.

II. Related Work

A. Indic Speech Technologies

Indian-language speech processing has moved forward quickly over the past few years, largely on the back of a handful of large multilingual resources. The Vistaar collection assembled diverse ASR datasets and baselines spanning Indian languages [1], and the model families that grew out of it — IndicWhisper and IndicConformer among them — are now competitive, openly released multilingual ASR systems [2], [3]; IndicConformer is, in fact, the speech-recognition backbone iTantra uses. On the synthesis side, IndicVoices contributed a large and genuinely inclusive multilingual speech dataset [4], later scaled by IndicVoices-R into one of the biggest multi-speaker corpora available for Indian TTS training [5]. Open-source efforts such as Vakyanish [6], along with more systematic comparisons of acoustic models and vocoders for Indian languages [7], show that VITS-style models [9] paired with efficient vocoders can produce natural, low-latency speech even with comparatively little training data. In practice, however, licensing and availability constraints meant iTantra ended up using established open voice-synthesis projects — Piper, Coqui, and Mimic3, described further in Section IV — rather than a custom Indic-trained voice for every language, and Section IV-C reports the resulting coverage gap plainly.

B. Ultra-Low-Bitrate Neural Speech Coding

A different line of research tackles the same bandwidth problem by compressing the speech signal itself rather than its meaning. LPCNet gets speech down to about 1.6 kb/s at a quality well beyond older parametric codecs [10]; SoundStream operates anywhere from 3 to 18 kb/s and matches or beats Opus at a fraction of the bitrate [11]; Lyra V2 pushes further still, with production-ready modes between 3.2 and 9.2 kb/s that outperform legacy telecom codecs [12]. Together, these show that a few kilobits per second is enough for genuinely natural speech — but all of them are still sending acoustic information. iTantra sends text instead, smaller by another one to two orders of magnitude, trading some acoustic nuance (speaker identity, emotional tone) for a payload that is dramatically smaller.

C. Offline Peer-to-Peer Mesh Communication

Several deployed systems tackle infrastructure-independent communication directly. FireChat, one of the earliest and most widely used, combined Bluetooth and peer-to-peer Wi-Fi to pass text messages from device to device [13]. Bridgefy works over Bluetooth with roughly 100 metres of range per hop and includes a mesh mode where messages jump through intermediate phones until they reach their destination; it has actually been used during earthquakes and periods of civil unrest, and claims more than 12 million downloads [14], [15]. Newer open-source projects such as BitChat go further, using Bluetooth Low Energy mesh with up to seven hops and end-to-end encryption [16]. What these systems share is multi-hop relaying — the sender and receiver don't need to be near each

other, because the devices in between forward the message. None of them, however, recognise or synthesise speech: they move text or files a person has typed, not spoken words, and none reconstruct voice in multiple languages the way iTantra aims to. It is also worth noting, and we return to this in Section IV, that despite occasionally being described as a mesh system, iTantra's current implementation connects exactly two devices directly and does not yet relay through intermediate peers.

D. Semantic Communication

A related but separate idea comes from semantic communication research, which argues that if a sender and receiver share the same underlying language model, they only need to exchange the meaning of a message — the receiver's own generative model can reconstruct an appropriate surface form [17]. iTantra fits this pattern once you look at speech through the same lens: the payload it sends is text, and the reconstruction step is an on-device, multilingual text-to-speech voice.

E. On-Device Language Models

Small, quantised language models capable of running entirely on a phone have matured rapidly, with models such as Microsoft's Phi-3-mini demonstrating strong instruction-following performance at a size (a few gigabytes, further compressed by 4-bit quantisation) that fits comfortably on modern mobile hardware via runtimes like llama.cpp [18]. iTantra uses exactly this combination for its companion assistant feature, discussed in Section IV-D, alongside a deliberately simple keyword-matching fallback for devices or situations where the full model isn't available.

III. Problem Formulation and Research Questions

Reliable, multilingual voice communication over links that are either severely bandwidth-constrained or simply absent is a well-recognised need across Indian space, disaster-management, and defence contexts, where narrowband radio and low-bitrate transmission are the rule rather than the exception. iTantra tackles this in its most immediately usable form — offline, ad-hoc communication between two smartphones — while keeping the door open to future integration with narrowband radio or satellite links. The rest of the paper is organised around one central question, with five more specific questions supporting it:

RQ1 (central): Can on-device speech-to-text and text-to-speech processing carry reliable voice communication over an extremely low-bandwidth peer-to-peer link, with no internet infrastructure at all?

RQ2: How much communication payload does this actually save, compared with sending conventional audio for the same spoken message?

RQ3: What end-to-end latency does the system achieve, from the moment someone starts speaking to when the reconstructed audio plays back on the other device?

RQ4: How accurately can Indian-language speech be recognised on resource-constrained mobile hardware, and how much does this vary from language to language?

RQ5: How much does background noise — a quiet room versus traffic, a crowd, or disaster-like conditions — affect recognition accuracy, given that the system currently relies on a simple energy-based speech detector rather than a neural one?

RQ6: How does Wi-Fi Direct actually compare with Bluetooth RFCOMM for this payload size and usage pattern, in terms of throughput, connection time, reliability, and range?

One clarification is worth making explicit here. The wire protocol carries both a source-language and a destination-language field, which might suggest the system translates between languages as part of a live conversation. The real-time voice pipeline does not: it performs multilingual speech recognition and multilingual speech synthesis, recognising a Marathi utterance and resynthesising it in Marathi, rather than turning it into Hindi at the other end. A separate, text-based assistant feature (Section IV-D) does perform translation using an on-device language model, but this is a distinct query-and-answer interaction, not part of the live speech transceiver. Folding translation into the real-time voice pipeline itself

remains future work, not a claimed capability of the current system.

IV. System Architecture

On the sending side, the microphone captures a 16 kHz mono PCM stream in 100 ms chunks, which are screened for speech activity and then buffered until roughly 800 ms of silence signals the end of an utterance (with a 30-second hard cap per message). The buffered audio goes to the on-device IndicConformer speech-recognition model for the selected language — each language is its own INT8-quantised ONNX graph, loaded lazily and cached after first use — and the resulting text, together with source and destination language tags, a device identifier, a timestamp, a sequence number, and a confidence score, is packed into a Protocol Buffers message and sent as a length-prefixed frame. On the receiving side, that message is decoded, the language tag selects the matching neural voice, and the synthesised audio plays back through the standard media audio stream. A separate ALERT message type is defined in the protocol for forcing maximum-volume, alarm-style playback with a Do-Not-Disturb bypass attempt, though as Section IV-C explains, there is currently no screen or button in the app that actually triggers one.

A. Network Layer: Wi-Fi Direct vs. Alternatives

Technology	Typical Range	Measured Throughput	Hops
Wi-Fi Direct (primary)	~50–200 m	~10–23 Mb/s (1 hop); ~6–17 Mb/s (2–3 hops)	1 (native); multi-hop needs custom relay
Wi-Fi Aware (NAN)	Comparable/greater	Higher than BT at longer range	1 (native)
Bluetooth RFCOMM (fallback)	~10–30 m	~1–2 Mb/s (1 hop), degrading per hop	Poor multi-hop scaling
BLE mesh	~10 m/hop	~1.4 Mb/s/hop	Designed for many hops

Even for payloads as small as iTantra's, Wi-Fi Direct offers roughly an order of magnitude more throughput than Bluetooth, which is why it takes the primary role in the implementation (over a plain TCP socket), with RFCOMM as a lower-power fallback when the Wi-Fi Direct connection cannot be established. Table I points to a genuine limitation, though: both transports connect exactly two devices directly. There is no relay logic in the current codebase, so despite the project occasionally describing itself with mesh terminology, it does

not yet behave like one. Systems built for the same disaster and field-operation environments — FireChat, Bridgefy, BitChat — route messages through intermediate devices precisely because the sender and receiver often aren't within direct range of each other during an actual disaster. As it stands, iTantra is best understood as direct-range, walkie-talkie-style communication, with multi-hop relay as a clear and worthwhile future direction rather than something the current design handles.

B. Wire Format: Protocol Buffers vs. Alternatives

Format	Size vs. JSON	Serialize Speed	Schema Required
JSON	baseline	slowest	No
MessagePack	~17% smaller	fastest to serialize	No
Protocol Buffers (used)	~46% smaller	fast, balanced	Yes
FlatBuffers	similar/larger (padding)	slower to serialize, zero-copy read	Yes

Given that each message is small, follows a fixed schema, and is sent once per utterance rather than in a tight loop, Protocol Buffers' mix of compact size, fast encoding and decoding, and strong typing makes it a sensible choice. FlatBuffers' main advantage, zero-copy reads, matters most for high-frequency or large payloads, neither of which describes this use case, and

schema-free formats such as JSON or MessagePack simply spend bytes the system can't afford on an already bandwidth-starved link.

C. Implementation Reality: What the Build Process Actually Found

Two findings from real, on-device development are worth reporting as results in their own right, since they say something

about where multilingual, offline neural speech systems are genuinely difficult, independent of any one team's execution.

The first concerns voice activity detection. The system ships with a Silero VAD v4 ONNX model, a well-regarded neural detector, but testing against silence, a pure sine tone, and white noise all produced the same near-zero speech probability, meaning the model did not discriminate speech from non-speech in this deployment configuration. Rather than fabricate an integration that did not actually work, the current build disables the neural path and falls back to a simple RMS-energy threshold, with the neural model kept loaded so it can be re-enabled once the underlying issue is understood. This is a meaningfully weaker detector — it will be more sensitive to background noise than a working neural VAD would be — and it directly motivates RQ5 and the noise-robustness experiment in Section VII.

The second concerns text-to-speech coverage. Of the ten Indian languages the project originally targeted, only five currently have a verified, freely licensed offline voice after checking the Piper, Coqui, Mimic3, and MMS voice catalogues: Hindi, Malayalam, and English through Piper, Bengali through Coqui, and Gujarati through Mimic3 (documented internally as a lower-quality voice, being the only source found). No usable free offline voice exists yet for Marathi, Kannada, Tamil, Telugu, or Odia, and Odia additionally lacks a speech-recognition model in the source repository used, leaving it with no coverage on either side. Where no voice exists, the system returns an explicit error rather than silently falling back to the wrong language or fabricated audio. We report this gap directly because it reflects a real, currently unsolved constraint in open, freely licensed Indic speech synthesis, not a limitation specific to this implementation.

D. On-Device AI Tactical Assistant

Alongside the real-time voice link, the app includes a separate assistant screen intended for first-aid and disaster-protocol queries. When the device supports it (64-bit ARM or x86_64 hardware) and an optional 2.4 GB model has been downloaded, queries are answered by Phi-3-mini-4k-instruct running locally through llama.cpp, with generation capped to a bounded number of tokens after early testing showed the model could otherwise continue well past a useful answer length. When the full model isn't available, a keyword-matching fallback covering the same nine languages answers instead, and the interface tells the user plainly which of the two is responding rather than presenting keyword matches as if they were generated text. This same assistant is capable of translating text between languages, which is where genuine cross-lingual capability currently lives in the system — as a deliberate, visible query rather than a hidden step inside the voice transceiver.

V. Use Cases and Beneficiaries

A. Disaster Response

When rescue teams move into an area with no GSM signal, no internet, and unreliable power, they can pair two devices over

Wi-Fi Direct and talk to each other in their own language. This targets the coordination problem that shows up whenever cellular infrastructure is the first thing to fail in a flood, earthquake, or cyclone, though as Section V-E notes, the emergency-broadcast side of this use case is currently only partially realised.

B. Tactical Field Operations

In remote or contested environments, where conventional network infrastructure either doesn't reach or is deliberately avoided, having offline, low-power, always-running peer-to-peer communication with a small transmission footprint is useful on its own, quite apart from any language consideration.

C. Rural and Remote Areas

In villages where GSM coverage is patchy at best, iTantra lets nearby smartphones coordinate locally, handle emergencies, and support community or field-worker communication, all without relying on centralised network infrastructure or a paid connection.

D. ISRO Mission Support

Mission-support communication under tight bandwidth constraints is a natural extension of the same architecture, though it hasn't been validated for that setting. We present it here as a potential application area, pending testing under conditions that actually resemble a mission, and in line with ISRO's broader interest in low-bitrate radio access.

E. Emergency Alerting: A Partially Realised Use Case

The wire protocol defines an ALERT message type, and the audio-playback layer already knows how to force maximum volume, switch to an alarm-style stream, and attempt to bypass Do-Not-Disturb settings. What does not currently exist is any screen or button in the app that a user can press to actually send one — the original five-screen design included a dedicated SOS/Emergency interface, and it was dropped during implementation. The underlying capability is real and tested at the playback level, but from a user's point of view, emergency alerting is not yet a feature of the shipped application. We list it here as a near-term, low-effort addition rather than an existing use case, since the hardest parts (the message type and the playback behaviour) are already in place.

F. Relevance to Indian Government Objectives

India's National Disaster Management Authority sets out guidelines for a National Disaster Management Information and Communication System that specifically call out last-mile connectivity in disaster-hit areas as a distinct requirement that remains underserved, and it asks for portable communication platforms and equipment for NDRF battalions to help close that gap [19]. iTantra's offline, infrastructure-independent, multilingual voice link is a reasonable candidate for exactly this kind of last-mile role: it needs no dedicated spectrum, no telecom operator involvement, and no ongoing cloud dependency (the internet is used once, for setup), and being built from open, auditable components fits well with a government preference for indigenous, inspectable technology

in sensitive deployments. For ordinary citizens, the practical payoff is that a standard Android phone becomes a working communication device the moment infrastructure goes down, in their own language rather than only English or Hindi — for five of the ten languages currently targeted, at least, with the remainder awaiting a usable open voice model. We want to be careful here: this is a policy-relevant application area, not a

claim that any government body has adopted or endorsed the system. Actually validating it would mean running field trials with disaster-response agencies, and closing the language-coverage and emergency-alerting gaps described above first.

VI. Comparison with Existing Approaches

Approach	Needs Internet/GSM	Bandwidth	Offline	Multilingual Voice
Cellular voice	Usually	High	No	Limited
Internet VoIP	Yes	Medium/High	No	App-dependent
Radio walkie-talkie	No	Radio-based	Yes	No language processing
Bluetooth voice apps	No	Limited	Yes	No language processing
Mesh apps (Bridgefy/FireChat/BitChat)	No	Very low (text)	Yes	No (text only, no voice synthesis)
iTantra	Once, for setup only	Very low (~50-300 B/utterance)	Yes (after setup)	9 languages recognised, 5 with verified voices

This comparison is where the actual research gap becomes clear. Offline mesh-messaging systems solve the infrastructure-independence problem, but only for typed text. Low-bitrate voice codecs solve the bandwidth problem, but they still send acoustic signal and have no language-processing layer built in. Nothing in this table combines on-device multilingual speech recognition, a compact linguistic payload, on-device speech synthesis, and infrastructure-independent peer-to-peer networking in one working system — and that combination, even with the coverage gaps documented in Section IV-C, is what iTantra actually contributes.

VII. Evaluation Plan

We've organised the evaluation around seven experiments, each tied to a research question from Section III or to a performance target already set out in the project specification.

The first measures bandwidth reduction: how many bytes it takes to send the same spoken sentence as raw audio versus as an iTantra Protobuf message, addressing RQ2.

The second measures recognition accuracy, tracking Word Error Rate for each of the nine supported speech-recognition languages against target thresholds (Hindi under 8%; averaged across all nine, under 18%), for RQ4.

The third looks at end-to-end latency, timing the gap from speech detection through recognition, transmission, and synthesis, to when playback actually starts on the receiver, against a target Real-Time Factor under 2.5 seconds, for RQ3.

The fourth compares the two network transports directly: transmission time, connection setup time, reliability, range, and reconnection time for Wi-Fi Direct versus Bluetooth RFCOMM under identical test conditions, for RQ6.

The fifth profiles resource consumption on-device — CPU, RAM, battery drain, and model load/inference time — against targets of under 80 MB idle RAM, under 5% idle CPU, and under 400 MB peak inference RAM, to check that the system is genuinely feasible on constrained mobile hardware.

The sixth tests noise robustness, tracking how much recognition accuracy degrades across quiet rooms, moderate background noise, traffic, crowds, and disaster-like acoustic conditions, for RQ5 — this matters more than it otherwise would, given that the currently active speech detector is energy-based rather than neural.

The seventh is new relative to the original plan: a direct comparison of the disabled neural VAD against the active RMS-energy detector on the same recorded audio, to quantify what is actually being given up by the fallback, and to establish a concrete target for re-enabling the neural path.

VIII. Advantages, Challenges, and Limitations

The main advantage of iTantra is that it separates achievable bandwidth from acoustic fidelity. Because text takes up orders of magnitude less space than audio, usable voice communication becomes possible over links, or device pairings, that couldn't sustain a real-time audio stream at all, and language coverage can grow simply as more training data or open voices become available. That said, the system has a number of real limitations, most of them documented directly in the codebase and worth stating plainly rather than glossing over. Recognition errors don't degrade gracefully the way an acoustic codec's errors do; they change the actual meaning of the reconstructed speech, which is a qualitatively different kind of failure. Speaker identity, emotional tone, and code-mixed or dialectal speech aren't preserved by a text-only channel. The active voice detector is energy-based rather than neural, after the bundled neural model was found not to discriminate speech from noise in testing, which likely makes the system more sensitive to background noise than intended. Text-to-speech coverage is five of the ten originally targeted languages, with one of those five documented as lower quality, and speech recognition itself has no working model for Odia. The architecture connects exactly two devices directly rather than relaying through a mesh, unlike the deployed disaster-communication systems built for similar environments. Real-time translation is not part of the voice pipeline; it exists only in the separate text-based assistant. There is no working

emergency-alert interface yet, despite the underlying message type and playback behaviour already existing. A per-peer chat screen in the app is a visual mock not yet wired to the real pipeline. Model downloads run on a plain background coroutine rather than a job scheduler, so an interrupted download does not resume automatically after the app process is killed. And finally, sending human-readable text rather than an opaque waveform raises privacy and security questions that the current design does not yet address through encryption.

IX. Conclusion and Future Work

This paper has framed iTantra's core contribution as low-bandwidth, infrastructure-independent, on-device multilingual speech communication, with the Android application as its working implementation and testbed — described here as it actually behaves, including where it fell short of its original plan. By replacing a continuous audio stream with a locally generated linguistic payload, the system aims for bandwidth reduction an order of magnitude or more beyond even the best current neural speech codecs, while supporting speech recognition in nine Indian languages and verified voice synthesis in five, entirely offline after a one-time setup download. A meaningful amount of work remains. On the engineering side: re-enabling or replacing the neural voice-activity detector, since the current energy-based fallback is a real step down in noise robustness; sourcing or training usable open voices for the five languages that currently have none; building the emergency-alert screen that the underlying message type and playback path already support; wiring the per-peer chat screen to the real pipeline instead of its current placeholder; and moving model downloads onto a job scheduler so they survive process death. On the research side: running the seven experiments in Section VII on physical two-device hardware; extending the network layer from direct pairing to genuine multi-hop relay, following the lead of systems like Bridgefy and BitChat; and, longer term, folding the translation capability that already exists in the on-device assistant into the real-time voice pipeline itself, so that a Marathi speaker and a Hindi listener could eventually use the system directly.

References

- [1] K. S. Bhogale, S. Sundaresan, A. Raman, T. Javed, M. M. Khapra, and P. Kumar, "Vistaar: Diverse benchmarks and training sets for Indian language ASR," in *Proc. Interspeech*, 2023, pp. 4384–4388.
- [2] AI4Bharat, "IndicConformer-600M-Multilingual: Conformer-based ASR for 22 Indian languages," Hugging Face model release, 2024.
- [3] AI4Bharat, "IndicWhisper: Whisper fine-tunes for Indian languages," GitHub repository, 2023.
- [4] T. Javed et al., "IndicVoices: Towards building an inclusive multilingual speech dataset for Indian languages," in *Findings of ACL*, 2024, pp. 10740–10782.
- [5] A. Sankar et al., "IndicVoices-R: Unlocking a massive multilingual multi-speaker speech corpus for scaling Indian TTS," in *NeurIPS Datasets and Benchmarks Track*, 2024.
- [6] H. Chadha et al., "Vakyansh: ASR toolkit for low resource Indic languages," *arXiv preprint*, 2022.

- [7] A. Prakash et al., "Towards building text-to-speech systems for the next billion users," *arXiv:2211.09536*, 2022.
- [8] A. Łańcucki, "FastPitch: Parallel text-to-speech with pitch prediction," in *Proc. ICASSP*, 2021.
- [9] J. Kim, J. Kong, and J. Son, "VITS: Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in *Proc. ICML*, 2021.
- [10] J.-M. Valin and J. Skoglund, "LPCNet: Improving neural speech synthesis through linear prediction," in *Proc. ICASSP*, 2019.
- [11] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "SoundStream: An end-to-end neural audio codec," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 30, pp. 495–507, 2021.
- [12] Google Research, "Lyra V2: A new generation in speech compression," *Google Research Blog*, 2022.
- [13] Open Garden, "FireChat: mesh networking for offline messaging," *product documentation*, 2014.
- [14] Bridgefy Inc., "Bridgefy offline messaging: Bluetooth mesh networking for crisis communication," *company documentation*, accessed 2026.
- [15] The Daily Star, "3 messaging apps that don't rely on internet connection," *technology feature*, 2025.
- [16] R. Puneet, "BitChat: Bluetooth LE mesh network with Noise Protocol encryption," *GitHub repository*, 2026.
- [17] Z. Qin, X. Tao, J. Lu, and G. Y. Li, "Semantic communications: Principles and challenges," *arXiv preprint*, 2021.
- [18] Microsoft, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv:2404.14219*, 2024.
- [19] National Disaster Management Authority, Government of India, "National Disaster Management Guidelines: National Disaster Management Information and Communication System (NDMICS)," *ndma.gov.in*, 2012.